Kurseinheit	Phase	Beschreibung	Theorie (Quelle) bzw. Praxis (Quelle)	Zeit Lehrer bzw. Schüler
		Leitidee: "Wie können wir Objekte mit einer Kamera entdecken, um damit einen ESP32 anzusteuern?"		
K1	Thema	Was sieht die Kamera?		
K1	Theorie	Pixel, Farbraum, Farbtiefe, RGB	Mit **Pixel**, Bildpunkt werden die einzelnen Farbwerte einer digitalen Rastergrafik bezeichnet sowie die zur Erfassung oder Darstellung eines Farbwerts nötigen Flächenelemente bei einem Bildsensor beziehungsweise Bildschirm mit Rasteransteuerung. (...) Die im Pixel verwendete **Kodierung** der Farbe definiert sich unter anderem über den Farbraum und die Farbtiefe. Der einfachste Fall ist ein Binärbild, bei dem ein Pixel einen Schwarzweiß-Wert speichert. (...) Ein **Farbraum** umfasst die Menge aller Farben, die durch eine farbverarbeitende Methode tatsächlich verwendet werden können. Die **Farbtiefe** bestimmt ... die Differenzierung aller Helligkeits- und Farbwerte. Am gebräuchlichsten ist der RGB-Farbraum mit 8 Bit pro Kanal ... (wikipedia)	
K1	Praxis	https://codemypic.de/	**CodeMyPic** ist ein freier, didaktisierter, quelltextbasierter Grafikeditor, dessen Zielsetzung es ist, Schülerinnen und Schülern einen Einblick in den Aufbau sowie die Funktionsweise von einfachen, ASCII-basierten Bildkodierungen zu ermöglichen. Unterstütztes Grafikformat (u. a.): Portable Bitmap (PBM) **Portable Bitmap Kopfdaten:** 1. Magischer Wert, 2 Byte: Format der Bilddaten: P1 oder P2 oder P3 2. Leerraum 3. Breite des Bilds in Pixeln (dezimal in ASCII kodiert) 4. Leerraum 5. Höhe des Bilds in Pixeln (dezimal in ASCII kodiert) 6. Leerraum 7. Maximalwert für die Helligkeit (nur bei P2 und P3) 8. Leerraum **Bilddaten** Bilddaten in Form von Helligkeitswerten der einzelnen Pixel bzw. für jeden Farbkanal eines Pixels, entweder dezimal in ASCII oder binär kodiert. (wikipedia)	

K1	Praxis	Zeichne eine Null	P1 #Bitmap # Eine Null 5 7 #Breite des Bildes, Leerstelle, Höhe des Bildes 0 1 1 1 0 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 1 0 0 0 1 0 1 1 1 0	
K1	Praxis	Farbbild	P3 # Ein Farbbild der Größe 3 × 2 Pixel, maximaler Helligkeit 255. # Darauf folgen die RGB-Tripel. 3 2 255 255 0 0 0 255 0 0 0 255 255 255 0 255 255 255 0 0 0	Bis hierhin: 10 Min.
K1	Aufgabe	Bild bauen "Schweizer Flagge"		15 Min
K1	Theorie	Überblick: H7 Plus Cam und OpenMV IDE	Die **OpenMV Cam** ist ein flexibles System zur Lösung einfacher Bildverarbeitungs-probleme zu entwickeln. Um dies zu erreichen, basiert die OpenMV Cam auf MicroPython, das auf einem Mikrocontroller läuft. Dieser Ansatz ermöglicht es, Python-Skripte zu schreiben, die auf der OpenMV Cam ausgeführt werden und extrem schnell kompiliert und geladen werden, während gleichzeitig die Vorteile eines Mikrocontrollers wie geringer Stromverbrauch und sofortige Verarbeitung für eingebettete Anwendungen erhalten bleiben. (...) Die **OpenMV IDE** ist das Werkzeug, mit dem du die OpenMV-Kamera programmierst. Die OpenMV IDE verfügt über einen leistungsstarken Texteditor auf Basis von Qt Creator, einen Framebuffer-Viewer, eine Histogrammanzeige und ein integriertes serielles Terminal für Debug-Ausgaben Ihrer OpenMV-Kamera. (...) (OpenMV Cam Tutorial)	

K1	Praxis	Hardware-Einrichtung	Verbinde die OpenMV-Kamera mithilfe eines **Micro-USB-Kabels** mit dem Computer und starte anschließend die OpenMV IDE. Bitte gehe **vorsichtig** mit dem Micro-USB-Kabel um (…) Starte anschließend die OpenMV IDE und klicke auf die **Schaltfläche „Verbinden“** (unten links in der OpenMV IDE). Die OpenMV IDE sollte sich dann automatisch mit der OpenMV Cam verbinden. (...) Die OpenMV Cam verfügt über ein **integriertes USB-Flash-Laufwerk.** Es wird angezeigt, sobald die OpenMV Cam an den Computer angeschlossen ist. (...) Du kannst den Zustand des internen Flash-Laufwerks Ihrer OpenMV Cam **versehentlich beschädigen**, indem du versuchst, *Bilder* auf dem internen Flash-Laufwerk zu speichern. (...) Verwende den internen Flash-Speicher nur zum **Speichern von Dateien**, die du manuell auf Ihre OpenMV Cam kopiert hast, während diese mit Ihrem Computer verbunden ist. (...) Verwende die **Funktion zum sicheren Entfernen von Hardware**, bevor du die OpenMV Cam aussteckst. Wenn du eine **microSD-Karte** in den microSD-Kartensteckplatz Ihrer OpenMV Cam einlegen, ersetzt die microSD-Karte den internen Flash-Speicher.	

			Im **Menü „Datei“** der OpenMV IDE findest du alle Standardoptionen eines Texteditors: Neu, Öffnen, Speichern, Speichern unter, Drucken usw. (…) OpenMV IDE verfügt über einen professionellen **Texteditor**, der auf dem QtCreator-Backend basiert. (…) Sobald du die Bearbeitung deines Codes abgeschlossen hast und bereit bist, ein Skript auszuführen, **verbinde** die Kamera mit der OpenMV IDE und klicke auf die **grüne Schaltfläche „Ausführen“** unten links in der OpenMV IDE. Das Skript wird dann an die OpenMV Cam gesendet, in Python-Bytecode kompiliert und von der OpenMV Cam ausgeführt. (...) Das Programm **helloworld_1.py** startet eine unendliche Schleife. In der Schleife wird ein Foto gemacht und das Bild ans Programm zurückgegeben. Das **serielle Terminal in der IDE** zeigt **Text-Output** der OpenMV Cam an. (...) Wenn du das Skript beenden möchtest, klicke auf die **Stopp-Schaltfläche**, die während der Skriptausführung die Ausführen-Schaltfläche ersetzt (...) Der **Framebuffer-Viewer** zeigt an, was sich zuvor im Framebuffer der OpenMV Cam befand, als csi.CSI.snapshot() aufgerufen wurde. (...) Die integrierte **Histogrammanzeige** in OpenMV IDE dient primär dazu, den leeren Bereich unter dem Framebuffer-Viewer zu füllen und dir eine ansprechende Darstellung zu bieten. Sie ist aber auch nützlich, um Feedback zur Lichtqualität im Raum zu erhalten, die Einstellungen für die Farbnachführung zu bestimmen und dir allgemein einen Eindruck von der Bildqualität Ihrer OpenMV-Kamera zu vermitteln.	
K1	Praxis	Mit der OpenMV IDE Foto machen und Ergebnis beobachten	Für **Input/ Output** nutze die Funktion **„Extras/ Terminal öffnen“** in der OpenMV IDE. Das ist ein Standard-Terminal für die Kamera. Alle Tastatureingaben werden an die Kamera gesendet und angezeigt, wenn sich die Kamera im REPL Modus befindet. (...)	Bis hierhin: 10 Min.
K1	Aufgabe	Fotos von verschiedenen Objekten machen	Bildqualität beurteilen: Belichtung, Schärfe, helle und dunkle Bildbereiche	10 Min
K2	Thema	Wie lernt eine Maschine?		

K2	Theorie	Neuronale Netze, Machine Learning, KI	**Neuronale Netze** sind ein Teil des **Machine Learnings**. (...) Eine **Künstliche Intelligenz** ist sogar ein wenig allgemeiner gehalten: Als KI bezeichnet man jedes Programm, das versucht, menschliches Verhalten zu imitieren. Machine Learning ist spezifischer und sagt: Ihr wollt ein Programm aus Daten Rückschlüsse ziehen lassen. Ihr wollt ähnliche Daten erkennen, ähnliche Daten produzieren oder zum Beispiel eine Klasse von etwas bestimmen: Ist das jetzt ein Hund, oder ist es eine Katze, ist es ein Zebrastreifen oder ein Fußgänger. (...) Ein Programm, das aus Daten herausfindet, welche Merkmale für eine Aufgabe am wichtigsten sind.(...) Diese Programme können je nach Datensatz, bei gleichem Code völlig unterschiedlich werden und ganz andere Aufgabe bewältigen. Doch Machine Learning Programme oder allgemein KI machen Fehler, wie wir Menschen auch. (...) Man gibt den Fehler häufig in Prozent an, z. B. 86 % aller Daten korrekt erkannt. Das Problem ist: Wir müssen das messen können. (...) (BWKI - ML Grundkurs - Einleitungsvideo)	
K2	Theorie	Starke KI vs. Schwache KI Trainingsdaten vs. Testdaten	Ein Beispiel für eine **starke KI** ist ein persönlicher Hilfsroboter (...), der sich ganz genau an deine Bedürfnisse anpassen kann. Google Assistant, Amazon Alexa, Apple Siri, usw. sind Beispiele für moderne, aber nichtsdestotrotz **“schwache” KIs**. Ein weiteres Paradebeispiel für eine schwache KI ist ein Algorithmus, welcher das Bild einer handschriftlichen Zahl in eine numerische Zahl übersetzen kann. (...) KI kann man grob einteilen in schwache und starke KI. Starke KI kann eigenständig Aufgaben erlernen und meistern, für die sie nicht geschaffen wurde, während hingegen schwache KI nur die Aufgabe lösen kann, für die sie programmiert wurde. Heutzutage existieren nur schwache KI-Systeme. Eine KI hat stets eine bestimmte Aufgabe, auf die sie trainiert wird. Beim **Training** werden dem KI-Modell viele Beispiele der zu bewätigenden Aufgabe gezeigt und die KI versucht bei jedem dieser Beispiele eine möglichst gute Lösunung zu bieten. Bei jedem neuen Beispiel wird anhand einer **Kostenfunktion** ermittelt, wie gut die KI die jeweilige Aufgabe bewältig hat.Nach dem Training wird eine KI getestet. Hierfür verwenden wir **Test-Daten**, die die KI nicht schon zuvor im Training gesehen hat, um herauszufinden, ob sie wirklich gelernt hat die Aufgabe zu lösen oder einfach nur die Trainingsdaten auswendig gelernt hat. (BWKI - Machine Learnig Grundkurs)	

K2	Theorie	Supervised vs. Unsupervised learning Klassifizierung vs. Regression	Wir unterscheiden zwischen Methoden des **supervised** und **unsupervised learning**. Beim supervised learning sind die Daten mit **labels** versehen, wohingegen beim unsupervised learning die Daten nicht mit labels versehen wurden und der Algorithmus selbstständig lernen muss, Muster im Datensatz zu erkennen. Bei einem **Klassifizierungsproblem** möchten wir, dass unsere KI eine Klasse oder eine Kategorie vorhersagt. (...) Im Kontrast dazu geht es bei einem **Regressionsproblem** darum, eine beliebig genaue Schätzung zu generieren. (...) Labels bei einem Klassifizierungsproblem nehmen bestimmte, feste (“diskrete”) Werte an, während hingegen Labels bei einem Regressionsproblem beliebige (“kontinuierliche”) Werte annehmen können. (BWKI - Machine Learning Grundkurs)	
K2	Theorie	Neuronale Netze	**Künstliche Neuronale Netzwerke** wurde nach dem Vorbild eines echten Gehirns entwickelt und basieren auf einem Netzwerk aus vielen Neuronen, die in einzelne Layer unterteilt werden und miteinander verknüpft sind. Beim **Training** führen wir immer wieder einen Forward und Backward Path für verschiedene Beispiele in unserem Datensatz durch. Beim Forward Path wird die **Vorhersage** des Modells berechnet, beim Backward Path nutzen wir Ableitungen, um die **Parameter** des Netzwerkes so **anzupassen**, dass der Fehler minimiert wird. Neuronale Netzwerke sind sehr flexibel einsetzbar und können für sehr viele verschiedene Aufgaben trainiert werden. Durch Hinzufügen von **mehr Neuronen und Layern** kann das Netzwerk komplexere Zusammenhänge lernen, braucht dann aber auch länger für das Training. (BWKI - Machine Learning Grundkurs)	
K2	Praxis	Einführung des Wollperzeptrons	Neuronale Netze bestehen aus miteinander verknüpften künstlichen Neuronen (**Perzeptrone**). Bei der Aktivität übernehmen die Schülerinnen bzw. Schüler die Aufgaben der Ein- und Ausgaben Beim **Wollperzeptron** übernehmen die Schülerinnen bzw. Schüler die Aufgaben der Ein- und Ausgaben und **simulieren** unter Zuhilfenahme von Wäscheklammern und Farbplättchen den **Lernprozess**. (Das Wollperzeptron – Eine Unplugged Aktivität zum Maschinellen Lernen)	Bis hierhin: 20 Min.

K2	Aufgabe	Das Wollperzeptron trainieren	**Augen-Gruppe:** Kann sehen, aber nicht denken. Kann vorher festgelegte Merkmale erkennen. **Gehirn-Gruppe:** Kann denken, aber nicht sehen. Sind Ausgabeneuronen. Jede/r hat eine Karte mit dem Ergebnis (0/1). (Einführungspräsentation) **Rollen** verteilen, das **Wollperzeptron** aufgebauen, **Anleitungen** lesen. **Training** beginnen. Dabei befinden sich zu Beginn keine Wäscheklammern auf den Wollfäden. Die **Spielleiterin** bzw. der Spielleiter steht hinter der Gehirn-Gruppe, damit diese die Bilder nicht sehen kann und wird nacheinander einzelne **Bilder** mit Sportarten **hochhalten**. Die Mitglieder der Augen-Gruppe **untersuchen** die Bilder hinsichtlich der ihnen zugeteilten Merkmale. Erkennen sie ihr Merkmal, so halten sie ein Schild mit dem **Eingabewert 1**, ansonsten mit dem **Eingabewert 0** hoch.Jedes Mitglied der Gehirn-Gruppe **empfängt** den Eingabewert der einzelnen Merkmale der Augen-Gruppe. (...) **Multiplikation**. (...) **Lernregel.** (...) (Das Wollperzeptron – Eine Unplugged Aktivität zum Maschinellen Lernen)	20 Min.
K2	Aufgabe	Bedeutung der Bestandteile des Wollperzeptrons	Bedeutung der einzelnen Gruppen, der Wollfäden und der Wäscheklammern ebenso wie die Vorgehensweise, die Berechnung des Skalarprodukts und die Deltaregel besprechen. (Das Wollperzeptron – Eine Unplugged Aktivität zum Maschinellen Lernen)	5 Min.
K3	Thema	KI-Modell trainieren		

K3	Theorie	Rolle von Edge Impulse Training Supervised Learning Overfitting	**Edge Impulse** ist eine Plattform, die den Prozess der Erstellung von Machine-Learning-Modellen **vereinfacht**, indem sie sinnvolle Standardwerte für die unzähligen Parameter auswählt, die bei der Erstellung eines ML-Modells festgelegt werden könnten. Sie bietet eine einfache Benutzeroberfläche, die es nicht nur ermöglicht, ein ML-Modell zu trainieren, sondern auch, die Daten zu inspizieren und das Modell zu testen. (...) Um ein ML-Modell für die Klassifizierung von Bildern zu **trainieren**, müssen wir es mit Bilddaten des entsprechenden Objekts **füttern**. Während des Trainingsprozesses wird das Modell mithilfe eines Konzepts geschult, das als „überwachtes Lernen“ (**Supervised Learning**) bezeichnet wird. Dies bedeutet, dass wir das Modell mit bekannten Daten trainieren und ihm – während es seine Vorhersagen „übt“ – mitteilen, ob diese korrekt sind oder nicht. (...) Ein Aspekt, den es zu berücksichtigen gilt, ist das **Overfitting**. Wenn ein Machine-Learning-Modell ein Overfitting aufweist, bedeutet dies, dass es zu stark auf deine Trainingsdaten abgestimmt ist und bei unbekannten Eingabedaten keine gute Leistung erbringen wird. (...) Du benötigst etwas Variation im Trainingsdatensatz und passt die Parameter so an, dass das Modell nicht einfach alle Eingabedaten auswendig lernt und darauf basierend die Klassifizierung vornimmt, sondern dass das Modell das Konzept eines Objekts lernen soll. (...) Die richtige Konfiguration für deine Anwendung zu finden, erfordert oft Ausprobieren. (...) (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	
K3	Theorie	Workflow	1. In OpenMV: Datensatz erstellen 2. In OpenMV: Daten nach Edge Impulse hochladen 3. In Edge Impulse: Daten erfassen 4. In Edge Impulse: Einen Impuls kreieren 5. In Edge Impulse: Features erzeugen 6. In Edge Impulse: Das Modell trainieren 7. In Edge Impulse: Das Modell testen 8. In Edge Impulse: Export nach OpenMV 9. In OpenMV: Das Python-Skript ausführen (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	Bis hierher: 10 Min.

		Edge Impulse Account kreieren	In Edge Impulse: Benutzername, E-Mail-Adresse, Passwort (merken !!), Bedingungen zustimmen, E-Mail empfangen, Account aktivieren -> Welcome
			Create New Project, Name eingeben, Personal, Create New Project.
			Target OpenMV H7 Plus auswählen
			Wenn später beim Hochladen der Daten nach Edge Impuls ein Authentifizierungs-Fehler gemeldet wird: Keys/ Add new API key: Development Key Warnung ignorieren
			
			In OpenMV IDE: Extras/ Datensatzeditor/ Neuer Datensatz: Neuen
			Ordner Phi_Psi_Omega erstellen und auswählen
			Bearbeite das ereits geöffnete Bildaufnahmeskript:
			Die Kamera steht auf dem Kopf. Füge deshalb hinter Zeile 12 ein:
			`csi0.vflip(True) # Flips the image vertically`
			`csi0.hmirror(True) # Mirrors the image horizontally`
			Wir **beginnen** mit dem Buchstaben Omega, weil die OpenMV IDE die Ordner alphabetisch geordnet anzeigt. In der Werkzeugleiste: Klicke auf das Symbol Neuer Klassenordner/ letter_Omega erstellen. Führe nun das Bildaufnahmeskript aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst.
			Fokussiere den Buchstaben Omega mit der Kamera und klicke auf das Symbol Daten erfassen, um ein Bild davon aufzunehmen. Wiederhole dies 30x. Ändere Abstand und Winkel zwischen den Aufnahmen.
			Ebenso: In der Werkzeugleiste ... letter_Phi ... Buchstabe Phi
			Ebenso: In der Werkzeugleiste ... letter_Psi ... Buchstabe Psi
			Ebenso: In der Werkzeugleiste ... unknown ... Buchstabe U oder leere Tafel
			(https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	

K3	Aufgabe	Fotos machen		10 Min.

K3	Aufgabe	Fotos zu Edge Impulse übertragen Daten erfassen Impuls kreieren	In **OpenMV IDE:** Extras/ Datensatzeditor/ Export/ In Edge Impulse hochladen/ Projekt wählen/ Trainingsdaten 80% Testdaten 20% -> 120 Dateien hinzugefügt In **Edge Impulse:** Öffne dein Projekt und navigiere zu **„Data Acquisition“**. Du siehst, dass die **Bilder** hochgeladen und entsprechend den von Ihnen erstellten Klassen **gelabelt** wurden. **Die Frage:** "Are you building an object detection project" muss mit NO beantwortet werden. Wenn du das nicht tut, löscht Edge Impuls die Labels. Abhilfe: Du kannst über Project info/ Project info/ Labeling method wieder "One label per data item" einstellen. Mit dem Data Acquisition Tool kannst du die Bildbeispiele durchsuchen und jene entfernen, die du für das Training als nicht wertvoll erachtest (z. B. wenn eines der Bilder zu unscharf ist). Wenn du mit den Datensamples zufrieden sind, kannst du mit der Gestaltung deines Impulses fortfahren. Ein Impuls ist – kurz gesagt – ein Rezept, anhand dessen das Modell trainiert wird. Er definiert Aktionen, die auf Ihre Eingabedaten angewendet werden, um diese für das maschinelle Lernen besser geeignet zu machen, sowie einen Lernblock, der den Algorithmus für die Klassifizierung festlegt. Navigiere im Menü unter **„Impulse Design“** zu **„Create Impulse“** und füge sowohl einen **Image Processing Block** als auch einen **Transfer Learning Block** hinzu. Klicke auf **„Save Impulse“**, um die angepassten Einstellungen zu übernehmen. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	10 Min.

K3	Aufgabe	Features erzeugen	In **Edge Impulse:** In diesem Schritt passt du die Bildeinstellungen an und generierst die Merkmale aus den Eingabedaten. Merkmale sind eindeutige Eigenschaften, die vom Klassifizierungsalgorithmus verwendet werden, um die Objekte zu erkennen. (...) Navigieren im Menü unter „Impulse Design“ zum Punkt **„Image“**. Stelle die Farbtiefe auf „RGB“ ein und **speichere die Parameter**. Klicke anschließend auf **„Generate Features“**. Der Analysevorgang wird – abhängig von der Anzahl der hochgeladenen Bilder – einige Zeit in Anspruch nehmen. Sobald er abgeschlossen ist, kannst du die Ergebnisse begutachten. Im Feature Explorer siehst du eine Visualisierung der Merkmale. Du kannst erkennen, dass einige Buchstaben (Punkte verschiedener Farbe) nur schwer voneinander zu unterscheiden sind, weil sich einige Datenpunkte in unmittelbarer Nähe zueinander befinden. Gibt es einen Buchstaben, der sich leichter abgrenzen lässt? (...) (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.

K3	Aufgabe	Modell trainieren	In **Edge Impulse:** Nun, da die Merkmale deiner Bilddaten bereitstehen, um für das eigentliche Training verwendet zu werden, kannst du im Menü zum Punkt **„Transfer Learning“** navigieren. Du musst die Einstellungen nun geringfügig anpassen. Legen die „Anzahl der Trainingszyklen“ auf einen Wert fest, der gute Ergebnisse liefert. In diesem Beispiel übernehem wir die Voreinstellung 20. Dies bestimmt, wie oft das Modell trainiert wird. Das Modell verbessert sich mit jedem Zyklus. (...) Wähle den **Model-Typ** MobileNetV2 96x96 0.35 (final layer: 16 neurons, 0.1 dropout) (...) Klicke auf **„Training starten“**, um das Machine-Learning-Modell zu trainieren. Eine kleine Anzahl von Bildern – das Validierungsset – wird beiseitegelegt, bevor das Training beginnt; dies dient der Validierung des trainierten Modells. Dies ist nicht mit dem Testset zu verwechseln, das zur Evaluierung des endgültigen Modells verwendet werden kann. Sobald das Training abgeschlossen ist, erhältst du Statistiken darüber, wie gut das Modell während der Validierung abgeschnitten hat. Idealerweise erreichst du für jedes Objekt eine Genauigkeit von 100 %. Solltest du schlechte Ergebnissez erhalten, enthält dein Datensatz möglicherweise Bilder, die für die zu klassifizierenden Objekte nicht repräsentativ sind und daher entfernt werden sollten. **Training output** zeigt, wie sich **accuracy** und **val_loss** mit jedem Zyklus verändern. Die **Confusion matrix** zeigt die Genauigkeit des ML-Modells nach dem letzten Trainingszyklus an. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.

K3	Aufgabe	Modell testen	In **Edge Impulse:** Nach dem Training des Modells hast du eine Vorstellung davon, wie gut das Modell auf den Daten abschneidet, die ihm aus dem Training bekannt sind. Dies ist jedoch nur die halbe Wahrheit. Du musst auch wissen, wie gut es auf bisher ungesehenen Daten performt. In fast jeder realen Anwendung wird ein Modell ausschließlich mit ungesehenen Daten konfrontiert. Die Fähigkeit, damit umzugehen, ist von entscheidender Bedeutung. Das Edge Impulse Studio stellt ein Tool bereit, mit dem du das Modell auf einfache Weise testen kannst. Du findest es unter dem Menüpunkt **„Model Testing“**. Die Ergebnisse des Modelltests geben dir Aufschluss über die Leistungsfähigkeit. Erzielt das Modell während des Tests schlechte Ergebnisse, wies jedoch nach dem Training eine hohe Genauigkeit auf, könnte dies auf ein Overfitting hindeuten. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification/)	5 Min.
K4	Thema	KI-Modell auf OpenMV Cam testen		
K4	Praxis	ML-Modell in Edge Impulse besprechen	In Edge Impulse: ML-Modell öffnen Create Impulse mit den 4 Blöcken Image mit den Processed Features Transfer Learning mit der Confusion Matrix und dem Data Explorer für beide KI-Modelle Model testing für das float32 Modell	5 Min.
K4	Aufgabe	Build OpenMV library Download der Modelle	In **Edge Impulse:** Das ML-Modell ist trainiert und bereits für den Einsatz auf Mikrocontrollern optimiert. Dies erfolgt automatisch im Hintergrund durch Quantisierung. Dabei handelt es sich um einen Prozess, bei dem der Wertebereich der Zahlen innerhalb der Machine-Learning-Modelle eingeschränkt wird, um die Leistung zu steigern – unter Inkaufnahme eines geringfügigen Genauigkeitsverlusts. Wechsle im Menü zum Bereich **„Deployment“**, wähle unter **„Configure your deployment“** die Option **„OpenMV library“** aus und klicke auf **„Build“**. Dadurch wird eine **Bibliothek** erstellt, die deinen Impuls auf der OpenMV Cam ausführt. Entpacke die Datei, sobald der Download abgeschlossen ist. Der Download sollte automatisch starten. (https://docs.arduino.cc/tutorials/nicla-vision/image-classification Die ZIP-Datei enthält nur das int8-Modell. So erhältst du das float32-Modell: Navigiere zu **Dashboard/ Download block output.** Klicke auf das Download-Symbol neben TensorFlow Lite (float32), um den Download zu starten.	10 Min.

K4	Aufgabe	Entpacken Umbenennen Editieren	Im Windows Datei-Explorer:
			Entpacke die Datei ei-<user>-<project>-openmv-v-<number>-impulse-#1.zip
			Benenne die Dateien um:
			Vorher: trained.tflite Nachher: trained_phi_psi_omega_int8.tflite
			Vorher: labels.txt Nachher: labels_phi_psi_omega.txt
			Vorher: ei_image_classification.py
			Nacher: ei_image_classification_phi_psi_omega_int8.py
			Vorher: ei-<user>-<project>tensorflow-lite-float32-model.8.lite
			Nacher: trained_phi_psi_omega_float32.lite
			
			Mit einem Editor:
			Editiere die Datei ei_image_classification_phi_psi_omega_int8.py
			Hinter Zeile 12 einfügen:
			`sensor.set_vflip(True)  # Flips the image vertically`
			`sensor.set_hmirror(True)  # Mirrors the image horizontally`
			Ersetze alle Vorkommen von labels.txt durch: labels_phi_psi_omega.txt
			Ersetze alle Vorkommen von trained.tflite durch: trained_phi_psi_omega_int8.tflite
			Speichern nicht vergessen !!
			Kopiere die geänderte Datei ei_image_classification_phi_psi_omega_int8.py nach:
			ei_image_classification_phi_psi_omega_float32.py
			Editiere die Datei ei_image_classification_phi_psi_omega_float32.py
			Ersetze alle Vorkommen von trained_phi_psi_omega_int8.tflite durch:
			trained_phi_psi_omega_float32.lite
				10 Min.

K4	Aufgabe	Anschließen Kopieren Testen	Im Windows Datei-Explorer: Schließe die Kamera an den Computer an. Das integrierte USB-Laufwerk wird als OpenMV D: angezeigt, Kopiere die Dateien vom Computer auf Laufwerk D: ei_image_classification_phi_psi_omega_float32.py ei_image_classification_phi_psi_omega_int8.py labels_phi_psi_omega.txt trained_phi_psi_omega_int8.tflite trained_phi_psi_omega_float32.lite In OpenMV IDE testen: Öffne die Datei: ei_image_classification_phi_psi_omega_int8.py Führe nun das KI-Modell aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben Phi mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Ändere Abstand und Winkel zum Buchstaben Phi. Dieses KI-Modell läuft mit ca. 13 Frames per Second. Stoppe das Skript und beobachte die Ausgabe. Im KI-Modell wird softmax für Mehrklassenvorhersagen verwendet. Die Summe aller von softmax erzeugten Ausgaben ist 1. Mit dem Speichern-Symbol speicherst du die Ausgaben in einer Log-Datei auf deinem Computer. Mit dem Wischen-Symbol löschst du die Ausgaben im Seriellen Terminal bevor du das KI-Model erneut ausführt. Wiederhole den Test mit der Datei ei_image_classification_phi_psi_omega_float32.py Dieses (große) Modell läuft mit ca. 3 Frames per Second.	15 Min.
K4	Aufgabe	https://netron.app/	In Netron.app: Öffne die Datei: trained_phi_psi_omega_int8.tflite Input und Output anschauen, int8, Dimensionen, Softmax Öffne die Datei: trained_phi_psi_omega_float32.lite Input und Output anschauen, float32, Dimensionen, Softmax	5 Min.
K5	Thema	KI-Modell verbessern		

K5	Theorie	Mögliche Ursachen für schlechte Ergebnisse	Wenn dein **Impuls schlechte Ergebnisse** liefert, könnten folgende Ursachen dafür verantwortlich sein: 1. Es sind **nicht genügend** Daten vorhanden. 2. Die Daten **unterscheiden** sich optisch von jenen Daten, die das Netzwerk zuvor gesehen hat. 3. Das Modell wurde **nicht ausreichend** trainiert. 4. Wenn dein neuronales Netz eine hohe Genauigkeit aufweist, das Modell bei neuen Daten jedoch schlecht abschneidet, könnte dies auf ein **Overfitting** hindeuten. 5. Die **Architektur** des neuronalen Netzes passt nicht optimal zu Ihren Daten. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance)	
K5	Theorie	Mehr Daten	Es sind **nicht genügend** Daten vorhanden. Neuronale Netze müssen Muster in Datensätzen erlernen – und je mehr Daten zur Verfügung stehen, desto besser. Du kannst auch die Fensterverschiebung* (bei „Create Impulse“) verringern, um eine stärkere Überlappung der Fenster zu erzeugen; dies führt jedoch nicht zu einer größeren Varianz in Ihrem Datensatz. **Mehr Daten sind daher stets vorzuziehen.** (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) **) Time-Series Data Augmentation beim Developer Plan nicht möglich*	Bis hierher: 10 Min.

K5	Aufgabe	Daten vom Testsatz zum Trainingssatz verschieben	Die Daten **unterscheiden** sich optisch von jenen Daten, die das Netzwerk zuvor gesehen hat. Dies tritt häufig auf, wenn das Gerät auf eine Weise verwendet wird, die du nicht in deinen Testsatz aufgenommen haben. Wenn dir dies im Testsatz oder während der Live-Klassifizierung auffällt, kannst du das entsprechende Sample in den Trainingssatz verschieben, indem du auf ⋮ klickst und die entsprechende Option auswählst. Achte darauf, das Label vor dem Training zu aktualisieren. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) Gehe zum Menüpunkt **„Model Testing“**. Suche ein Sample (rot), das falsch klassifiziert wurde. Merke dir die Bildnummer des Samples. Gehe zum Menüpunkt **„Data Acquisition“**. Gehe in den Testdatensatz und verschiebe das Sample mit der Bildnummer, indem du auf die drei Punkte klickst und die entsprechende Option auswählst. Genauso verschiebst du ein anderes Sample vom Trainingsdatensatz in den Testdatensatz. Wiederhole anschließend die Schritte **„Generate Features“, „Training starten“, „Model Testing“.** Vergewissere dich, dass das Sample im Testsatz fehlt.	
K5	Aufgabe	Anzahl der Trainingszyklen erhöhen	Das Modell wurde **nicht ausreichend** trainiert. Erhöhe die Anzahl der Trainingszyklen und prüfe, ob sich die Leistung verbessert. Sollte kein Unterschied feststellbar sein, verfügst du wahrscheinlich über zu wenige Daten, oder die Daten lassen sich nicht gut genug voneinander trennen. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) Gehe zum Menüpunkt **„Transfer Learning“**. **Erhöhe** die "Number of training cycles". Wiederhole anschließend die Schritte **„Training starten“, „Model Testing“.** Prüfe, ob sich die Ergebnisse verbessert haben.	

K5	Aufgabe	Lernrate verringern	Wenn dein neuronales Netz eine hohe Genauigkeit aufweist, das Modell bei neuen Daten jedoch schlecht abschneidet, könnte dies auf ein **Overfitting** (Überanpassung) des Modells hindeuten. Es hat die Merkmale in Ihrem Datensatz zu stark verinnerlicht. Versuche, weitere Daten hinzuzufügen, oder verringere die Lernrate. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) Gehe zum Menüpunkt **„Transfer Learning“**. **Erniedrige** die "Learning rate". Wiederhole anschließend die Schritte **„Training starten“, „Model Testing“.** Prüfe, ob sich die Ergebnisse verbessert haben.	
K5	Aufgabe	Anzahl der Neuronen erhönen	Die Architektur des neuronalen Netzes passt nicht optimal zu Ihren Daten. Experimentiere mit der Anzahl der Schichten und Neuronen, um zu prüfen, ob sich die Leistung dadurch verbessert. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) Gehe zum Menüpunkt **„Transfer Learning“**. **Erhöhe** die "Number of Neurons" im Modell MobileNetV2 . Wiederhole anschließend die Schritte **„Training starten“, „Model Testing“**. Prüfe, ob sich die Ergebnisse verbessert haben.	
K5	Aufgabe	Datensatz neu ausbalancieren	**Klassenungleichheit** Wenn du in einer Klasse deutlich mehr Daten hast als in den anderen – beispielsweise, wenn 90 % Ihrer Daten als „idle“ und nur 10 % als „wave“ gelabelt sind –, wird dein neuronales Netz aufgrund der Klassenungleichheit Schwierigkeiten beim Lernen haben. In diesem Fall besteht die beste Lösung darin, die Datenmenge in der unterrepräsentierten Klasse zu erhöhen. Sollte dies jedoch nicht möglich sein, kannst du versuchen, deinen Datensatz während des Trainings neu auszubalancieren. Gehe hierfür zu deinem Lernblock und aktiviere die entsprechende Option (diese ist möglicherweise nicht für alle Lernblock-Typen verfügbar). (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance) Gehe zum Menüpunkt **„Transfer Learning“**. Klappe "Advanced training settings" aus. **Setze den Haken** bei "Auto-weight classes". Wiederhole anschließend die Schritte **„Training starten“, „Model Testing“.** Prüfe, ob sich die Ergebnisse verbessert haben.	Bis hierher: 30 Min.

K5	Theorie	Maßnahmen, die Abhilfe schaffen können	**Großer Unterschied** zwischen der Leistung des **quantisierten** (int8) und des **unoptimierten** (float32) **Modells**. Quantisierung funktioniert, indem sie die Präzision der Modellgewichte reduziert; daher kommt es häufig zu einer leichten Einbuße bei der Leistung. Sollte die Leistung zu stark abfallen, findest du hier einige Maßnahmen, die Abhilfe schaffen können: **Füge weitere Daten hinzu.** Es ist besonders wahrscheinlich, dass es zu Leistungseinbußen bei jenen Klassen kommt, die im Trainingsdatensatz in der Minderheit sind. Wenn du mit Zeitreihen arbeitest (beispielsweise von Beschleunigungssensoren), versuche, die Schrittweite des gleitenden Fensters zu verringern; dies erzeugt eine größere Anzahl an Samples. **Füge eine Form der Regularisierung hinzu** (beispielsweise durch Dropout-Schichten). Dies trägt dazu bei, das Netzwerk widerstandsfähiger gegenüber jener Art von Fehlern zu machen, die durch die Quantisierung entstehen. Möglicherweise musst du das Modell über einige zusätzliche Epochen trainieren, um die Auswirkungen der Regularisierung auszugleichen. **Erhöhe die Kapazität des Netzwerks.** Quantisierungsfehler wirken sich bei Netzwerken, die bereits an ihrer Kapazitätsgrenze arbeiten – d. h. bei denen jeder einzelne Parameter von entscheidender Bedeutung ist –, besonders gravierend aus. Wenn du dem Netzwerk einige zusätzliche Neuronen oder Schichten hinzufügst, wirst du möglicherweise feststellen, dass sich das Problem dadurch deutlich abschwächt. (https://docs.edgeimpulse.com/knowledge/guides/increasing-model-performance)	5 Min.
K6	Thema	ESP32 Ansteuerung		

K6	Theorie	Versorgungsspannung und digitale Pegel	Die **OpenMV Cam H7 Plus** wird mit **3.6 – 5 V** versorgt. Die **Versorgungspannung** wird an die Pins VIN und GND angeschlossen. Wenn sowohl USB als auch VIN/LiPo anliegen, hat der VIN/LiPo-Eingang Vorrang – der integrierte Stromwahlschalter wählt diesen gegenüber USB aus, um das Board mit Strom zu versorgen. Die OpenMV Cam H7 Plus hat 10 I/O-Pins, 5-V-tolerant mit 3,3-V-Ausgang, (...) P6 ist im ADC- oder DAC-Modus nicht 5-V-tolerant. Andere OpenMV Cams Einige OpenMV Cams verfügen über 5-V-tolerante I/O-Pins (ein 5-V-Signal kann ohne Beschädigung direkt angelegt werden); **andere** betreiben ihre I/O-Schnittstellen mit 3,3 V oder 1,8 V und benötigen für Signale mit höherer Spannung einen Pegelwandler. Der Anschluss einer 5-V-Quelle an einen nicht toleranten Pin kann den Chip beschädigen. (https://docs.openmv.io/openmvcam/tutorial) Die **ESP32-Platine** wird über USB mit **5 V versorgt**. Die ESP32-Platine verfügt über digitale Ein-/Ausgangs-Pins (GPIO Pins - General Purpose Input/Output Pins). Diese digitalen Eingänge/Ausgänge arbeiten mit 3,3V. Es darf keine 5V Spannung an die ESP32 Chip Pins angeschlossen werden! Die Pins sind nicht 5V-tolerant, das Anlegen von mehr als 3,3V an einem Pin wird den Chip beschädigen. (AZ-Delivery: ESP-32 Dev Kit C V4_DE.pdf) **Schlussfolgerung**: Wir können die OpenMV Cam direkt an den ESP32 anschließen.	5 Min.
K6	Aufgabe	Verdrahtung der UART-Kommunikation zwischen OpenMV Cam und ESP32	OpenMV Cam und ESP32-Platine von der **Spannungsversorgung trennen !!** Verbinde den OpenMV GND Pin mit dem ESP32 GND Pin. Verbinde den OpenMV Cam UART3 TX Pin (P4) mit dem ESP32 RX2 Pin (GPIO16) Verbinde den OpenMV Cam UART3 RX Pin (P5) mit dem ESP32 TX2 Pin (GPIO17) **Begründung:** Auf der OpenMV Cam arbeiten wir mit UART3. Auf der ESP32-Platine arbeiten wir mit UART2.	5 Min.

K6	Aufgabe	Ausführung des Programms ei_image_classification_phi_psi_omega_int8_serial_output.py	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Öffne das Programm ei_image_classification_phi_psi_omega_int8_serial_output.py. Das Programm ist eine Erweiterung des Programms, mit dem wir in den letzten beiden Stunden gearbeitet haben. Das Programm macht eine Ausgabe, wenn Softmax für ein Label einen Wert größer als den Schwellwert 0,5 liefert. In der Ausgabe steht das erkannte Label und die laufende Nummer des Schnappschusses. Nur nach jedem 10ten Schnappschuss wird eine Ausgabe gemacht, damit wir sie gut beobachten können. Die Strings werden durch Leerzeichen voneinander getrennt. Das ist notwendig, damit der Empfänger Anfang und Ende der Strings erkennen kann. print() schreibt die Ausgabe in das Serielle Terminal. uart.write() schreibt die Ausgabe auf den UART3. Während das Programm läuft, leuchtet die LED blau. Der Schwellwert kann geändert werden. Ebenso kann geändert werden, nach wie vielen Schnappschüssen eine Ausgabe gemacht wird. Führe nun das Programm aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben Phi mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Bewege die Tafel mit dem Buchstaben hin und her und beobachte die Ausgabe im Seriellen Terminal.	

K6	Aufgabe	Ausführung des Programms ESP32_UART_control_input_only.py	Schließe die ESP32-Platine über den zweiten USB-Anschluss an den Computer an. Starte die **Thonny IDE**. Öffne das Programm **ESP32_UART_control_input_only.py.** **uart.read()** empfängt ein **Bytes Object** über UART2. Wenn die empfangene Nachricht **nicht None** ist, versucht das Programm die Folge von Bytes zu dekodieren. Die **Exception** fängt den Unicode Error ab, der beim Stopp des OpenMV Cam Programm entsteht. sleep(5) sorgt dafür, dass das Byte Objekt nur alle 5 Sekunden gelesen wird. Dieser Wert kann **geändert** werden. Es simuliert, dass der ESP32 langsamer liest als die OpenMV Cam Ausgaben macht. Durch das langsamere Lesen enthält das Bytes Objekt eine Reihe von Labeln und Nummern. Nun soll das Programm nur das letzte Label und die letzte Nummer auswerten. Dazu spalten wir das Bytes Objekt in eine Liste mit Labeln und Nummer auf. Danach können wir die beiden letzten Elemente der Liste zusammenbauen und dekodieren. Führe nun das Programm aus und beobachte die Ausgaben in der ESP32 Shell. Bewege die Tafel mit dem Buchstaben hin und her und vergewissere dich, dass das letzte Label angezeigt wird.	Bis hierher: 10 Min.

K6	Aufgabe	OpenMV Cam ohne IDE betreiben	In einer Anwendung wird die OpenMV Cam ohne IDE betrieben. Dupliziere das Programm ei_image_classification_phi_psi_omega_int8_serial_output.py mit dem Windows Datei-Explorer auf dem USB-Laufwerk der OpenMV Cam. Es wir als OpenMV D: angezeigt, Lösche die Originaldatei main.py oder benenne sie um. Benenne das Duplikat in main.py um. Trenne die OpenMV Cam vom Computer. Verbinde den OpenMV VIN Pin mit dem ESP32 5V Pin der ESP32-Platine. Nun bootet die OpenMV Cam und schreibt die Ausgaben auf den UART3. Während das Programm läuft, leuchtet die LED blau. Beobachte die Ausgaben in der ESP32 Shell.	10 Min.
K6	Aufgabe	Verdrahtung der parallelen Kommunikation zwischen OpenMV Cam und ESP32	Die schnellste Kommunikation geht über den parallelen Output der OpenMV Cam. Mit 2 Pins kannst du 4 Zustände unterscheiden. Das ist ausreichend für die 4 Labels unseres KI-Modells. OpenMV Cam und ESP32-Platine von der **Spannungsversorgung trennen !!** Verbinde den OpenMV GND Pin mit dem ESP32 GND Pin. Verbinde den OpenMV Cam Pin P0 (PB15) mit dem ESP32 Pin 18 (GPIO18) Verbinde den OpenMV Cam Pin P1 (PB14) mit dem ESP32 Pin 19 (GPIO19)	5 Min.

K6	Aufgabe	Ausführung des Programms ei_image_classification_phi_psi_omega_int8_parallel_output.py	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Öffne das Programm ei_image_classification_phi_psi_omega_int8_parallel_output.py. Das Programm ist eine Erweiterung des Programms, mit dem wir in den letzten beiden Stunden gearbeitet haben. Das Programm macht eine Ausgabe, wenn Softmax für ein Label einen Wert größer als den Schwellwert 0,5 liefert. In der Ausgabe steht das erkannte Label und die laufende Nummer des Schnappschusses. Nur nach jedem 10ten Schnappschuss wird eine Ausgabe gemacht, damit wir sie gut beobachten können. print() schreibt die Ausgabe in das Serielle Terminal. Das Programm macht aus dem Label String einen 2 Bit Code, der als Status der parallelen Output Pins ausgegeben wird. Während das Programm läuft, leuchtet die LED blau. Der Schwellwert kann geändert werden. Ebenso kann geändert werden, nach wie vielen Schnappschüssen eine Ausgabe gemacht wird. Führe nun das Programm aus, indem du die OpenMV IDE mit der Kamera verbindest auf die Wiedergabetaste klickst. Fokussiere nun den Buchstaben Phi mit der Kamera und beobachte die Ausgaben am Seriellen Terminal. Bewege die Tafel mit dem Buchstaben hin und her und beobachte die Ausgabe im Seriellen Terminal.	

K6	Aufgabe	Ausführung des Programms ESP32_parallel_input.py	Schließe die ESP32-Platine über den zweiten USB-Anschluss an den Computer an. Starte die **Thonny IDE.** Öffne das Programm **ESP32_parallel_input.py.** Die **parallelen Input Pins des ESP32** bekommen den **Status** der parallelen Output Pins der OpenMV Cam. Aus beiden Bits wird ein Label Code gemacht. Anschließend wird der Label Code dekodiert. sleep(5) sorgt dafür, dass der Status nur alle 5 Sekunden gelesen wird. Dieser Wert kann geändert werden. Es simuliert, dass der ESP32 langsamer liest als die OpenMV Cam Ausgaben macht. Anders als bei der seriellen Kommunikation gibt es bei der parallelen Kommunikation keinen "alten" Status. Der ESP32 liest immer den aktuellen Status. Führe nun das Programm aus und beobachte die Ausgaben in der ESP32 Shell. Bewege die Tafel mit dem Buchstaben hin und her und vergewissere dich, dass das akuelle Label angezeigt wird.	Bis hierher 10 Min.
K7	Thema	Kreis erkennen		
K7	Theorie	Bildverarbeitung	Unter Bildverarbeitung versteht man in der Informatik und der Elektrotechnik die Verarbeitung von Signalen, die Bilder repräsentieren, beispielsweise Fotografien oder Einzelbilder aus Videos. Das Ergebnis einer Bildverarbeitung kann wiederum ein Bild sein oder auch eine Menge von Merkmalen des Eingabebildes. (wikipedia)	
K7	Theorie	Hough-Transformation	Die Hough-Transformation (Sprechweise [hʌf]) ist ein robustes globales Verfahren zur Erkennung von Geraden, Kreisen oder beliebigen anderen parametrisierbaren geometrischen Figuren in einem binären Gradientenbild, also einem Schwarz-Weiß-Bild, nach einer Kantenerkennung. (...) (wikipedia) Die Circle-Hough-Transformation (CHT) ist ein grundlegendes Verfahren zur Merkmalsextraktion in der digitalen Bildverarbeitung, das zur Erkennung von Kreisen in unvollkommenen Bildern eingesetzt wird. Die Kreiskandidaten werden durch ein „Voting“-Verfahren im Hough-Parameterraum sowie die anschließende Auswahl lokaler Maxima in einer Akkumulatormatrix ermittelt. Sie stellt eine Spezialisierung der Hough-Transformation dar. (wikipedia-en)	

K7	Theorie	Circle-Hough-Transformation	In einem zweidimensionalen Raum lässt sich ein Kreis wie folgt beschreiben: $(x - a)^2 + (y - b)^2 = r^2$ (1) wobei (a, b) der Mittelpunkt des Kreises und r der Radius ist. Ist ein Punkt (x, y) in 2D festgelegt, so lassen sich die Parameter gemäß Gleichung (1) bestimmen. Der Parameterraum ist dann dreidimensional: (a, b, r). Alle Parameter, die (x, y) erfüllen, liegen auf der Oberfläche eines umgekehrten rechtwinkligen Kegels mit der Spitze (x, y, 0). (...) (wikipedia-en)	
K7	Theorie	Bestimmung der Parameter	Bestimmung der Parameter bei **bekanntem Radius R** Ist der Radius fest vorgegeben, reduziert sich der Parameterraum auf zwei Dimensionen (die Position des Kreismittelpunkts). Jeder Punkt (x, y) auf dem ursprünglichen Kreis definiert gemäß Gleichung (1) einen Kreis mit dem Radius R und dem Mittelpunkt (x, y). Der Schnittpunkt all dieser Kreise im Parameterraum entspricht dem Mittelpunkt des ursprünglichen Kreises. Betrachte vier Punkte auf einem Kreis im Originalbild (links). Rechts ist die Kreis-Hough-Transformation dargestellt. Dabei wird vorausgesetzt, dass der Radius bekannt ist. Für jeden der vier Punkte (weiße Punkte) im Originalbild lässt sich im Hough-Parameterraum ein Kreis mit dem Mittelpunkt (x, y) und dem Radius r definieren. Zur Erfassung der Schnittpunkte wird eine Akkumulatormatrix verwendet. Im Parameterraum wird für jeden Kreis der Zählerstand (die Stimmenzahl) an denjenigen Stellen um eins erhöht, durch die der neu definierte Kreis verläuft. Anschließend lässt sich das lokale Maximum bestimmen (der rote Punkt in der Mitte der rechten Abbildung). Die Position (a, b) dieses Maximums entspricht dem Mittelpunkt des ursprünglichen Kreises. (...) Bestimmung der Kreisparameter bei **unbekanntem Radius** Da der Parameterraum dreidimensional ist, ist auch die Akkumulatormatrix dreidimensional. Man kann die **möglichen Radien durchlaufen** und für jeden Radius das zuvor beschriebene Verfahren anwenden. Schließlich werden die lokalen Maxima in der 3D-Akkumulatormatrix ermittelt. (...) (wikipedia-en)	Bis hierher: 20 Min.

K7	Aufgabe	Zielscheibe aufnehmen	Schließe die OpenMV Cam über USB an den Computer an. Starte die OpenMV IDE. Verbinde die OpenMV IDE mit der Kamera.
			Öffne Datei/Beispiele/ Feature Detection/ find_circles.py. Speichere find_circles_1.py auf dem integrierten USB-Flash-Laufwerk (Laufwerk D:).
			Ändere das Programm find_circles_1.py:
			Füge hinter Zeile 19 ein:
			`csi0.vflip(True) # Flips the image vertically`
			`csi0.hmirror(True) # Mirrors the image horizontally`
			Hinter der Zeile:
			`for c in img.find_circles(`
			Passe `threshold` an. Beginne mit `threshold=5000`
			Passe `r_max` und `r_min` an. Beginne mit `r_max=60`
			Hinter der Zeile:
			`img.draw_circle(c, color=(255, 0, 0))`
			Ergänze:
			`print("min =", c[1] - c[2])`
			`print("max =", c[1] + c[2])`
			Fokussiere nun die **Zielscheibe** mit der Kamera und beobachte das Bild im **Framebuffer-Viewer**. Bei der passenden Einstellung wird der Umriss der Zielscheibe mit einem **roten Kreis** markiert.
			Am **Seriellen Terminal** werden die **Koordinaten** des Kreis-Mittelpunkts, der **Radius** und die **Magnitude** (Konfidenzwert) angezeigt. Teste den Einfluss von Abstand und Beleuchtung der Zielscheibe. Dieses Programm läuft mit ca. 40 Frames per Second.
				10 Min.

K7	Aufgabe	RGB-Wert entnehmen	Um die Farben der Ringe zu erkennen, entnehmen wir einen **RGB-Wert** aus jedem Ring. Wir berechnen die **Koordinaten** der Mitte der Ringe mit Hilfe des Radius' und der Koordinaten des Mittelpunkts. Nur die y-Koordinate wird geändert. Die Angaben `min` und `max` am Seriellen Terminal zeigen uns, ob wir richtig liegen. Die Programmzeile: `r_brightness, g_brightness, b_brightness = img.get_pixel([c[0], y_coord])` liefert uns das RGB-Tupel.	5 Min.

K7	Aufgabe	Farben der Ringe erkennen	Nun müssen wir noch die Farbe des Rings erkennen und die zugehörige Zahl ausgeben. Diese Arbeit übernimmt die **Funktion** `my_number()`.
			Unsere Zielscheibe enthält die Farben (Tupel):
			Gelb (255, 255, 0)
			Cyan (0, 255, 255)
			Rot (255, 0, 0)
			Schwarz (0, 0, 0)
			Grün (0, 255, 0)
			Wir können nicht erwarten, dass die Kamera genau diese RGB-Tupel liefert. Deshalb müssen wir den Helligkeitswert des R - G- B-Kanals mit einem Schwellwert vergleichen:
			`r_status = False`
			`g_status = False`
			`b_status = False`
			`if r_brightness > r_threshold:`
			`  r_status = True`
			`if g_brightness > g_threshold:`
			`  g_status = True`
			`if b_brightness > b_threshold:`
			`  b_status = True`
			Zuerst testen wir den Schwellwert 127 für alle Kanäle. Später verbessern wir die Werte.
			Die Boole'schen Variablen `r_status, g_status, b_status` werten wir wie folgt aus:
			Gelb (True, True, False)
			Cyan (False, True, True)
			Rot (True, False, False)
			Schwarz (False, False, False)
			Grün (False, True, False)
			Die Funktion gibt dann die zugehörige **Zahl** zurück. Wenn keine Farbe erkannt wird, gibt die Funktion eine ungültige Zahl zurück.
				10 Min.